

A Reinforcement Learning Approach to Blackjack Strategy Under Casino-Realistic Conditions

1st Lucas Becker

*Ann and H.J Smead Aerospace Engineering
University of Colorado
Boulder, USA
lucas.becker@colorado.edu*

2nd Theodore Kishner

*Ann and H.J Smead Aerospace Engineering
University of Colorado
Boulder, USA
theo.kishner@colorado.edu*

Abstract—This paper presents a Markov Decision Process (MDP) formulation of the card game blackjack and investigates the application of reinforcement learning techniques to derive optimal playing policies under casino-realistic conditions. While prior work has largely treated blackjack as a simplified, infinite-shoe problem, modern casino blackjack is played with six to eight standard 52-card decks dealt from a shoe, with reshuffling occurring well before deck exhaustion. Faithfully modeling this finite, depleting deck structure introduces significant state space complexity that many formulations ignore.

We evaluate our approach across three levels of increasing fidelity and complexity. First, we establish baseline policies, a dealer-mirroring strategy that always hits on or below 17. Without allowing splitting or doubling, this provides a baseline performance which all learned policies are compared against. Second, we consider the single-deck setting, where the remaining card composition is tracked explicitly as counts of each card value still in the shoe. The MDP state is represented as the tuple (player hand total, dealer upcard, usable ace, remaining card counts), and Q-learning is applied to derive a near-optimal policy. Third, we extend the formulation to the full six-to-eight deck casino setting by augmenting the state space with a Hi-Lo true count feature, capturing the remaining deck composition in a single tractable integer. The resulting high-dimensional state space motivates the use of a deep Q-network (DQN), which approximates the action-value function with a neural network rather than an explicit table, enabling generalization across deck compositions that the agent has not explicitly visited during training.

Across all levels, policies are evaluated by win rate against a dealer following standard casino rules. Our results aim to demonstrate that deck-aware state representations meaningfully improve policy performance over deck-agnostic baselines, and that DQN offers a scalable path to near-optimal play in the full multi-deck casino setting.

I. INTRODUCTION

The game of blackjack has long served as a testbed for probabilistic modeling and decision theory. Since the seminal work of Baldwin et al. in 1956, which established the “basic strategy” using early statistical methods [5], the game has been recognized as one of the few casino offerings where player agency and mathematical rigor can meaningfully influence outcomes. At its core, blackjack presents a sequential decision-making problem under uncertainty, making it an ideal candidate for representation as a Markov Decision Process (MDP) [1]. In an MDP framework, a player (the agent) must choose an action – hitting, standing, doubling, or splitting – based

on a specific state to maximize an expected reward, typically defined by the final hand outcome relative to the dealer.

Traditional analyses often simplify the game by assuming an “infinite shoe” or immediate reshuffling, where the probability of drawing any card remains constant regardless of previous hands [4]. While this approach simplifies the state space, it fails to capture the dynamic reality of modern casino play. In practice, blackjack is played with a finite shoe of six to eight decks, and cards are dealt until a “cut card” is reached, often deep into the shoe. This depletion means that the transition probabilities between states are non-stationary; the probability of drawing a ten-valued card changes as they are removed from the deck. Leveraging this information through card-counting techniques has historically been the primary method for players to gain an edge, yet translating these heuristics into a rigorous computational policy remains a significant challenge [2], [7].

Recent advancements in reinforcement learning (RL) offer a robust framework for addressing these complexities. Unlike static strategy tables, RL agents can learn optimal behaviors through repeated interaction with a simulated environment [6]. For a single-deck setting, it is computationally feasible to track the exact composition of the shoe within the MDP state [3]. However, as the number of decks increases and the state space grows exponentially, tabular methods like standard Q-learning become intractable. This necessitates a transition to function approximation methods, such as Deep Q-Networks (DQN), which can generalize over a high-dimensional state space to identify patterns in deck composition that traditional heuristics might overlook.

This report investigates the efficacy of deck-aware reinforcement learning policies across varying levels of environmental fidelity. By progressing from a simple dealer-mirroring baseline to a DQN-based agent utilizing Hi-Lo true count features, we aim to quantify the performance gains associated with modeling the finite nature of the shoe. The following sections detail the MDP formulation for each complexity level, the training architectures employed, and a comparative analysis of the resulting policies against standard casino rules.

II. BACKGROUND AND RELATED WORK

The mathematical study of blackjack is historically rooted in the search for a "perfect" player strategy that minimizes the house edge. This section outlines the evolution of these strategies – from static statistical tables to dynamic reinforcement learning agents – and establishes the theoretical framework used in this project.

A. The Foundations of Blackjack Strategy

The seminal analysis of blackjack was conducted by Baldwin et al. in 1956. They derived the first "basic strategy" by evaluating the player's mathematical expectation for various hand totals against the dealer's upcard. This work demonstrated that because the dealer's actions are fixed and deterministic, the player's problem can be solved by identifying the action that maximizes the expected value of the hand. However, this early work largely assumed an infinite-deck environment where card probabilities remain constant, a simplification that persists in many modern educational examples.

B. Blackjack as a Markov Decision Process

To solve blackjack using modern computational methods, the game is typically framed as a Markov Decision Process (MDP). An MDP is defined by the tuple (S, A, T, R, γ) , where:

- S (States): The set of all possible game configurations, such as the player's hand total and the dealer's visible card.
- A (Actions): The decisions available to the agent, typically $\{\text{hit}, \text{stand}, \text{double}, \text{split}\}$.
- T (Transitions): The probability distribution over next states $P(s'|s, a)$, which in blackjack is determined by the composition of the remaining deck.
- R (Rewards): The immediate payoff, often 1 for a win, -1 for a loss, and 0 for a push.

Kochenderfer et al. provide a comprehensive framework for solving such sequential decision-making problems under uncertainty [1]. In the "infinite shoe" case, T is stationary. However, in a finite-deck setting, the removal of cards means the environment is technically a partially observable system unless the entire history of the shoe is tracked.

C. Reinforcement Learning Approaches

Reinforcement learning (RL) allows an agent to learn optimal policies directly through interaction with the environment without requiring a prior model of the transition probabilities T .

- Tabular Methods: Early RL applications to blackjack utilized Monte Carlo methods and Q-learning to approximate the action-value function $Q(s, a)$ [8]. These approaches are effective for simple state spaces, such as the standard 3-tuple representation (player total, dealer card, usable ace), but struggle when the state is expanded to include card-tracking features.
- Deck-Aware Learning: Previous technical reports have attempted to incorporate deck composition into the state space to mimic card-counting techniques. Finn explored

model-free Q-learning for a single-deck setting, while others have noted that even simple reinforcement learning agents can derive basic strategy independently by exploring the rewards of various hit/stand decisions [3].

- Deep Reinforcement Learning: When modeling a full casino shoe (6-8 decks), the state space becomes too large for a lookup table. Deep Q-Networks (DQN) solve this by using neural networks to approximate the Q-function, a method that has proven successful in high-dimensional gaming environments. This project builds on these concepts by using the Hi-Lo "true count" as a feature to compress the information of a multi-deck shoe into a tractable state representation.

By synthesizing the historical statistical foundations of Baldwin with modern deep learning architectures as discussed by Kochenderfer, this project explores whether a "deck-aware" agent can consistently outperform traditional deck-agnostic baselines in realistic casino conditions.

III. PROBLEM FORMULATION

We model blackjack as a Markov Decision Process (MDP) defined by the tuple $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, T, R, \gamma \rangle$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, T is the transition function, R is the reward function, and $\gamma = 1.0$ is the discount factor. The formulation is implemented across three levels of increasing fidelity, each extending the state space, action space, and simulation realism of the previous level.

A. Action Space

At Levels 1 and 2, the action space is restricted to the two core blackjack decisions:

$$\mathcal{A}_{1,2} = \{\text{hit}, \text{stick}\} \quad (1)$$

At Level 3, two additional casino-standard actions are introduced:

$$\mathcal{A}_3 = \{\text{hit}, \text{stick}, \text{double}, \text{split}\} \quad (2)$$

These actions are conditionally available. Doubling is permitted only on the player's first action of a hand, and splitting is permitted only when the player's first two cards are of equal value. After either action is taken, both flags are set to false for all subsequent states in the episode. Action masking enforces these constraints during both training and evaluation by setting Q-values of illegal actions to $-\infty$ before action selection.

B. State Space

The state space varies across the three solution levels and is the central design challenge of this work. In all cases, a state $s \in \mathcal{S}$ is marked as terminal once the round concludes, either by the player busting, by the player sticking and the dealer completing their hand, or by the player receiving a natural blackjack.

1) *Level 1 – Baseline (Deck-Agnostic)*: For the baseline policies, no learning occurs and no deck information is required. The decisions depend only on the minimal tuple:

$$s = (\text{player_sum}, \text{dealer_upcard}, \text{usable_ace}, \text{terminal}) \quad (3)$$

where $\text{player_sum} \in \{1, \dots, 31\}$ is the player’s current hand total, $\text{dealer_upcard} \in \{1, \dots, 10\}$ is the dealer’s visible card, $\text{usable_ace} \in \{\text{true}, \text{false}\}$ indicates whether the player holds an ace currently counted as 11 without busting, and $\text{terminal} \in \{\text{true}, \text{false}\}$ indicates whether the episode has ended. The effective number of decision-relevant states is approximately $21 \times 10 \times 2 = 420$.

2) *Level 2 – Single Deck with Q-Learning*: For the single-deck formulation, we augment the state with an explicit representation of the remaining shoe. A standard single deck contains 52 cards across 10 distinct values: $\{A, 2, 3, 4, 5, 6, 7, 8, 9, 10/J/Q/K\}$, with the 10-value group containing 16 cards and all others containing 4. The deck state is represented as a tuple of remaining counts:

$$\text{deck_counts} = (n_1, n_2, \dots, n_{10}) \in \{0, \dots, 4\}^9 \times \{0, \dots, 16\} \quad (4)$$

The full state is then:

$$s = (\text{player_sum}, \text{dealer_upcard}, \text{usable_ace}, \text{deck_counts}, \text{terminal}) \quad (5)$$

The theoretical upper bound on the number of distinct deck states is:

$$|\mathcal{S}_{\text{deck}}| \leq 5^9 \times 17 \approx 10^7 \quad (6)$$

However, since cards are only removed and never returned, the reachable portion of this space is far smaller in practice. Card probabilities are computed directly from the remaining deck counts at each state:

$$P(\text{draw } c) = \frac{n_c}{\sum_{i=1}^{10} n_i} \quad (7)$$

This gives the agent an exact, non-stationary distribution over the next card draw that updates as cards are removed from the shoe, faithfully modeling sampling without replacement from a finite deck.

Although the full deck state governs transition probabilities, the Q-table is indexed by the compact key:

$$q_key(s) = (\text{player_sum}, \text{dealer_upcard}, \text{usable_ace}) \quad (8)$$

This reduces the Q-table to at most $21 \times 10 \times 2 = 420$ entries per action, ensuring that every state-action pair is visited repeatedly during training and that Q-value estimates converge. The deck counts are retained in the full state and continue to govern transition probabilities, so the agent’s experience during training is shaped by the true finite deck distribution – only the generalization structure of the Q-table is simplified.

3) *Level 3 – Multi-Deck (6 Decks) with Deep Q-Network*: Extending the single-deck formulation to a six-deck shoe increases the count range dramatically. Each non-ten card value has at most 24 copies and the ten-value group has at most 96, giving a deck state space on the order of:

$$|\mathcal{S}_{\text{deck}}^6| \leq 25^9 \times 97 \approx 10^{14} \quad (9)$$

This is intractable for a tabular method. We therefore replace the explicit deck count vector with a single scalar feature, the Hi-Lo *true count* τ , defined as:

$$\tau = \frac{\text{running count}}{\text{decks remaining}} \quad (10)$$

where the running count is updated by +1 for each low card (2–6) seen, 0 for neutral cards (7–9), and –1 for each high card (10, J, Q, K, A). The true count normalizes for the number of decks remaining in the shoe, making it a consistent measure of deck favorability regardless of shoe depth. The running count is maintained across episodes within the same shoe and reset to zero whenever the shoe is reshuffled.

The Level 3 state also incorporates the expanded action space through two boolean flags indicating action availability. The full state representation is:

$$s = (\text{player_sum}, \text{dealer_upcard}, \text{usable_ace}, \text{can_double}, \text{can_split}, \tau, \text{terminal}) \quad (11)$$

Additionally, the state tracks several fields used internally for reward computation and transition logic:

$$s_{\text{internal}} = (\text{is_blackjack}, \text{doubled}, \text{first_card}, \text{deck_counts}) \quad (12)$$

The *is_blackjack* flag is set when the player’s initial two cards sum to 21, which triggers an immediate terminal state and a 3:2 payout. The *doubled* flag records whether the player doubled down, which doubles the reward magnitude at episode end. The *first_card* field records the value of the player’s first card and is used to determine split eligibility.

Rather than indexing a Q-table, the DQN takes s as a 6-dimensional continuous feature vector and approximates the action-value function $Q(s, a; \theta)$ with a neural network parameterized by weights θ . The input features are normalized: player_sum is divided by 21, dealer_upcard by 10, boolean flags are encoded as 0 or 1, and τ is divided by 10 and clamped to $[-1, 1]$. This allows the agent to generalize across deck compositions and action-availability combinations that were not explicitly visited during training.

C. Transition Function

The transition function $T(s, a, s') = P(s' | s, a)$ is defined separately for each action.

a) *Hit.*: A card c is drawn from the remaining deck according to the distribution in Equation (7). The player sum and usable ace flag are updated: if the drawn card is an ace and adding 11 does not cause a bust, the ace is counted as 11 and usable_ace is set to *true*; otherwise the ace is counted as 1. If after adding any card the player sum exceeds 21 and a

usable ace is held, the ace is downgraded from 11 to 1 and the sum is decremented by 10. After a hit, both `can_double` and `can_split` are set to `false`. The resulting state is terminal if and only if the player sum exceeds 21.

b) *Stick.*: The dealer plays out their hand according to the fixed casino rule: draw until the dealer sum is at least 17, then stop. The dealer's outcome distribution is computed analytically by recursing over all possible sequences of dealer draws weighted by their probabilities under the remaining deck counts. Each possible dealer final sum maps to a terminal state with probability equal to the likelihood of that draw sequence.

c) *Double (Level 3 only).*: Available only when `can_double = true`. The player draws exactly one additional card from the remaining deck, and the hand immediately terminates. If the player busts, the state is terminal with a doubled loss. Otherwise, the dealer plays out their hand analytically and the final state is terminal with the `doubled` flag set to `true`, which doubles the reward magnitude. The entire double-then-dealer-resolution is computed as a single compound transition.

d) *Split (Level 3 only).*: Available only when `can_split = true`. The player's pair is separated and the current hand is continued with one of the split cards. Next a new hand is started with one of the pair cards plus one newly drawn card from the deck. To avoid the complexity of tracking two simultaneous hands, we model splitting as beginning a fresh hand from the split card. The new hand has `can_double = true` but `can_split = false`, disallowing re-splitting as a simplification.

D. Reward Function

The reward function $R(s, a, s')$ is zero at all non-terminal transitions, with the outcome resolved only when the episode ends. At Levels 1 and 2:

$$R(s, a, s') = \begin{cases} -1 & \text{if player_sum} > 21 \quad (\text{bust}) \\ +1 & \text{if dealer busts or player_sum} > \text{dealer_sum} \\ 0 & \text{if player_sum} = \text{dealer_sum} \quad (\text{push}) \\ -1 & \text{if player_sum} < \text{dealer_sum} \end{cases} \quad (13)$$

At Level 3, the reward function is extended with two additions. First, a natural blackjack (21 on the initial two cards, before any action is taken) pays 3:2 rather than the standard 1:1:

$$R_{\text{blackjack}} = +1.5 \quad \text{if } \text{is_blackjack} = \text{true} \quad (14)$$

Second, if the player doubled down, all non-zero rewards are multiplied by 2:

$$R_{\text{double}}(s, a, s') = 2 \cdot R_{\text{base}}(s, a, s') \quad \text{if } \text{doubled} = \text{true} \quad (15)$$

The blackjack bonus is not doubled even if the hand originated from a doubled state, consistent with standard casino rules where natural blackjack only occurs on the initial deal.

E. Shoe Management

A persistent shoe is maintained across episodes at all three levels. Rather than resetting to a full deck at the start of every episode, the deck state at the end of one episode carries

forward as the starting deck state of the next. The shoe is reshuffled only when the fraction of remaining cards falls below a penetration threshold ρ :

$$\text{reshuffle if } \frac{\sum_{i=1}^{10} n_i}{52 \cdot D} < 1 - \rho \quad (16)$$

where D is the number of decks and $\rho = 0.65$, meaning the shoe is reshuffled when approximately 35% of the cards have been dealt. At Level 2, $D = 1$ and the shoe contains 52 cards. At Level 3, $D = 6$ and the shoe contains 312 cards. This penetration depth is consistent with standard casino practice, where six-deck shoes are typically cut at 60–75% penetration.

F. Initial State Distribution

At Levels 1 and 2, the episode begins by drawing the dealer's upcard from the current shoe according to Equation (7) and removing it from the deck. The player sum is initialized to 0 and the player receives cards via the hit transition.

At Level 3, the initial state distribution is computed by dealing three cards from the current shoe: one dealer upcard and two player cards. All $10 \times 10 \times 10 = 1000$ possible dealing sequences are enumerated with their exact probabilities under the current deck composition. For each initial deal, the state flags are set as follows: `is_blackjack = true` if the two player cards sum to 21, `can_split = true` if both player cards share the same value, and `can_double = true` for all hands. Natural blackjacks are immediately terminal

IV. SOLUTION APPROACH

A. Q-Learning for the Single-Deck Setting

We apply tabular Q-learning to derive an optimal hit/stick policy for the single-deck blackjack MDP. Q-learning is an off-policy temporal difference method that iteratively estimates the action-value function $Q(s, a)$, representing the expected discounted return from taking action a in state s and following the greedy policy thereafter. The optimal policy is then recovered as:

$$\pi^*(s) = \arg \max_{a \in A} Q(s, a) \quad (17)$$

1) *State Representation and Q-Table Design.*: A key implementation decision is the separation of the state used for transition dynamics from the state used to index the Q-table. As described in Section II, each `BJState` carries the full `deck_counts` tuple, which governs the card draw probabilities during simulation. However, indexing the Q-table on the full deck state would result in an intractable number of unique keys – nearly every state visited during training would be novel, and the Q-table would never converge. We therefore define a compact key function:

$$\text{q_key}(s) = (\text{player_sum}, \text{dealer_upcard}, \text{usable_ace}) \quad (18)$$

This reduces the Q-table to at most $21 \times 10 \times 2 \times 2 = 840$ entries (420 states $\times$ 2 actions), ensuring that every state-action pair is visited repeatedly during training and that Q-value estimates converge. The deck counts are retained in the full `BJState`

and continue to govern transition probabilities, so the agent’s experience during training is still shaped by the true finite deck distribution – only the generalization structure of the Q-table is simplified.

2) *Exploration Strategy*: We use an ε -greedy exploration strategy, where the agent selects a random action with probability ε and the greedy action $\arg \max_a Q(q_key(s), a)$ otherwise. The exploration rate decays over training according to:

$$\varepsilon_i = \max \left(0.1, 1 - \frac{2i}{N} \right) \quad (19)$$

where i is the current episode index and N is the total number of training episodes. This schedule decays at twice the rate of a standard linear schedule, reaching the floor of $\varepsilon = 0.1$ at the midpoint of training rather than at the end. This aggressive early decay was chosen because the compact 420-state Q-table is visited densely within the first half of training, and the remaining episodes benefit more from exploitation to refine Q-value estimates than from continued broad exploration.

3) *Q-Table Update Rule*: At each step within an episode, the Q-table is updated via the standard temporal difference rule:

$$Q(k, a) \leftarrow Q(k, a) + \alpha \left[r + \gamma \max_{a'} Q(k', a') - Q(k, a) \right] \quad (20)$$

where $k = q_key(s)$ and $k' = q_key(s')$ are the compact keys for the current and next states respectively, r is the immediate reward, $\alpha = 0.2$ is the learning rate, and $\gamma = 1.0$ is the discount factor. Unvisited state-action pairs are initialized to $Q = 0$ on first access via a default dictionary lookup.

4) *Shoe Management During Training*: Rather than resetting to a full deck at the start of every episode, training uses a persistent shoe that carries deck state across episodes. The shoe is reshuffled only when the fraction of remaining cards falls below a threshold $\rho = 0.65$, meaning a reshuffle occurs when approximately 35% of the deck has been dealt – consistent with casino penetration practices. This ensures that the agent trains on the realistic card distribution of a mid-shoe deck rather than always starting from a freshly shuffled deck.

5) *Best Policy Selection*: The Q-table is evaluated periodically during training every $E = 100$ episodes on a set of 10,000 greedy-policy rollouts. This evaluation count was chosen to match the final evaluation and ensure the statistically best Q-table is returned. The Q-table snapshot achieving the highest mean return across all evaluation checkpoints is saved and returned as the final policy, rather than simply using the last Q-table at the end of training. This guards against performance degradation late in training if residual exploration causes the agent to overwrite well-converged Q-values.

6) *Training Configuration*: Training is run for $N = 100,000$ episodes with learning rate $\alpha = 0.2$ and discount factor $\gamma = 1.0$. Progress is logged every $N/10 = 10,000$ episodes. The total number of unique Q-table entries at convergence is at most 840, and training completes in under one minute on a standard machine.

7) *Pseudocode*: Algorithm 1 summarizes the full training procedure.

Algorithm 1 Q-Learning for Single-Deck Blackjack

```

1: Initialize  $Q(k, a) \leftarrow 0$  for all  $k \in \mathcal{S}_{\text{compact}}, a \in \mathcal{A}$ 
2: Initialize shoe  $\mathcal{D} \leftarrow \text{fresh\_shoe}(1)$ 
3:  $\text{best\_Q} \leftarrow Q, \text{best\_return} \leftarrow -\infty$ 
4: for episode  $i = 1, \dots, N$  do
5:    $\varepsilon \leftarrow \max(0.1, 1 - 2i/N)$ 
6:   if  $\text{should\_shuffle}(\mathcal{D})$  then
7:      $\mathcal{D} \leftarrow \text{fresh\_shoe}(1)$ 
8:   end if
9:   Sample  $s_0 \sim \text{build\_initial\_dist}(\mathcal{D})$ 
10:   $s \leftarrow s_0$ 
11:  while  $s$  is not terminal do
12:    With prob.  $\varepsilon$ :  $a \leftarrow \text{rand}(\mathcal{A})$ 
13:    Otherwise:  $a \leftarrow \arg \max_{a'} Q(q\_key(s), a')$ 
14:    Sample  $s' \sim T(s, a)$  using  $\text{deck\_counts}$  in  $s$ 
15:     $r \leftarrow R(s, a, s')$ 
16:     $k \leftarrow q\_key(s), k' \leftarrow q\_key(s')$ 
17:     $Q(k, a) \leftarrow Q(k, a) + \alpha [r + \gamma \max_{a'} Q(k', a') - Q(k, a)]$ 
18:     $s \leftarrow s'$ 
19:  end while
20:   $\mathcal{D}.\text{counts} \leftarrow s.\text{deck\_counts}$  ▷ Sync shoe to
  end-of-episode deck
21:  if  $i \bmod E = 0$  then
22:     $\bar{r} \leftarrow \text{evaluate}(Q, 10,000 \text{ rollouts})$ 
23:    if  $\bar{r} > \text{best\_return}$  then
24:       $\text{best\_return} \leftarrow \bar{r}$ 
25:       $\text{best\_Q} \leftarrow \text{copy}(Q)$ 
26:    end if
27:  end if
28: end for
29: return  $\text{best\_Q}$ 

```

B. DQN for 6 Deck MDP

To address the 10^{14} state-space complexity of the multi-deck environment, we implement a Double Deep Q-Network (DDQN) using the Flux.jl library.

1) *Network Architecture*: The policy is parameterized by a feed-forward neural network with four layers: an input layer (6 features), two hidden layers of 128 neurons each, a third hidden layer of 64 neurons, and an output layer (4 actions). All hidden layers utilize ReLU activation functions.

2) *State Representation*: The agent receives a normalized 6-dimensional feature vector:

$$S_{\text{vec}} = \left[\frac{\text{sum}}{21}, \frac{\text{upcard}}{10}, \text{usable_ace}, \text{can_double}, \text{can_split}, \tau \right] \quad (21)$$

The Hi-Lo true count (τ) is clamped to $[-1, 1]$ to ensure stable gradients.

3) *Training Mechanics*: We employ an experience replay buffer with a capacity of 500,000 transitions to break temporal correlations. The network is optimized using the Adam optimizer with a learning rate of 5×10^{-5} and a batch size of 256.

4) *Action Masking and Double DQN*: To handle conditional actions (splitting and doubling), we implement action masking; Q-values for illegal actions are set to $-\infty$ during both action selection and the calculation of target values. We use a separate target network, updated every 50 steps, to provide stable learning targets for the Bellman equation.

5) *Exploration*: An ϵ -greedy strategy is used, with ϵ decaying linearly from 1.0 to a floor of 0.01 over the first 90% of the 100,000 training episodes.

V. RESULTS

We evaluate each policy over 10,000 independent episodes using a persistent shoe with the same penetration threshold used during training ($\rho = 0.65$). Mean return is reported alongside standard error to quantify statistical significance. A mean return of 0 corresponds to breaking even against the dealer; negative values reflect the house edge.

A. Level 1: Baseline Policies

The baseline policy is evaluated to establish performance, it does not use any deck information. It is evaluated using the mean return over 10,000 episodes.

TABLE I
BASELINE POLICY PERFORMANCE (SINGLE-DECK, 10,000 EPISODES)

Policy	Mean Return	Std Error
Stick on 17+	-0.0886	0.0094

The stick-on-17 policy mirrors the dealer’s fixed strategy and serves as the primary baseline against which learned policies are compared. Its mean return of -0.0886 reflects the inherent house edge under this simple rule.

B. Level 2: Q-Learning (Single Deck)

The Q-learning agent is trained for $N = 100,000$ episodes on a single-deck shoe and evaluated using the best Q-table checkpoint selected during training.

TABLE II
SINGLE-DECK POLICY COMPARISON (10,000 EPISODES)

Policy	Mean Return	Std Error
Baseline (Stick on 17+)	-0.0886	0.0094
Q-Learning (best checkpoint)	-0.0757	0.0095

The Q-learning agent improves upon the baseline by approximately 1.3 percentage points in mean return. While both policies remain negative due to the fundamental house edge, the learned policy demonstrates that even a compact 420-state Q-table trained on a deck-aware simulator can extract meaningful strategic improvements over a fixed rule. The learning curve in Fig. 1 shows that the agent converges within the first 10,000 steps in the environment, with marginal gains thereafter.

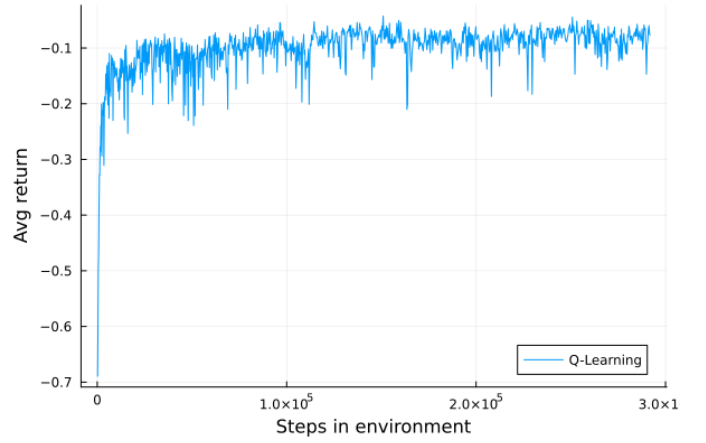


Fig. 1. Q-learning convergence on the single-deck MDP. Mean return is evaluated every 100 episodes over 10,000 greedy rollouts and plotted against cumulative environment steps.

C. Level 3: DQN (6-Deck Shoe)

The DQN agent is trained for 100,000 episodes on a 6-deck shoe with the Hi-Lo true count augmenting the state vector. The best network checkpoint is selected during training and used for final evaluation.

TABLE III
6-DECK POLICY COMPARISON (10,000 EPISODES)

Policy	Mean Return	Std Error
Baseline (Stick on 17+)	-0.1192	0.0092
DQN (Hi-Lo, 6-deck shoe)	-0.0781	0.0107

The DQN agent improves upon the 6-deck baseline by approximately 4.1 percentage points in mean return, a larger margin than the Q-learning improvement in the single-deck setting. This suggests that the Hi-Lo true count feature provides actionable information about deck favorability that the network learns to exploit. Notably, the 6-deck baseline performs worse than its single-deck counterpart (-0.1192 vs. -0.0886), consistent with the well-established result that increasing the number of decks raises the house edge.

The learning curve in Fig. 2 shows steady improvement over training.

The policy learned by the DQN algorithm is shown in Fig. 3 and Fig. 4. These figures represent the learned policy at a low count and a high count. Each color corresponds to an action that would be taken at that combination of cards presented. Green is hit, red is stay, yellow is double, and blue is split. When the score is higher, the algorithm views the deck as more favorable, the agent is deciding to double their bet more than the case of a lower score. The algorithm also learned to be more aggressive when the player has a useable ace in the hand, which is to be expected.

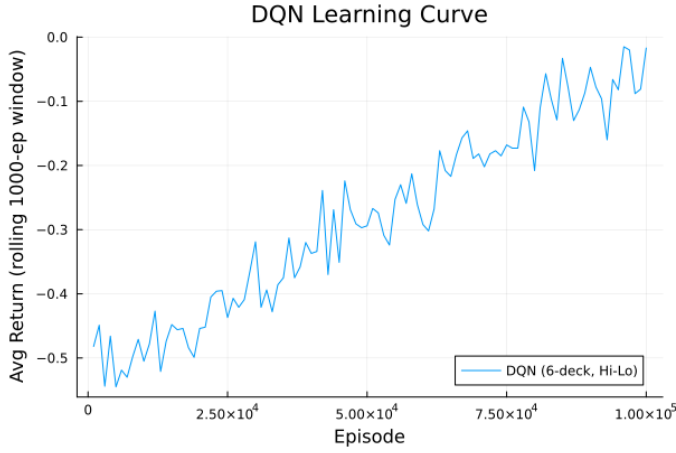


Fig. 2. DQN learning curve on the 6-deck blackjack MDP. Rolling average return is computed over 1,000-episode windows.

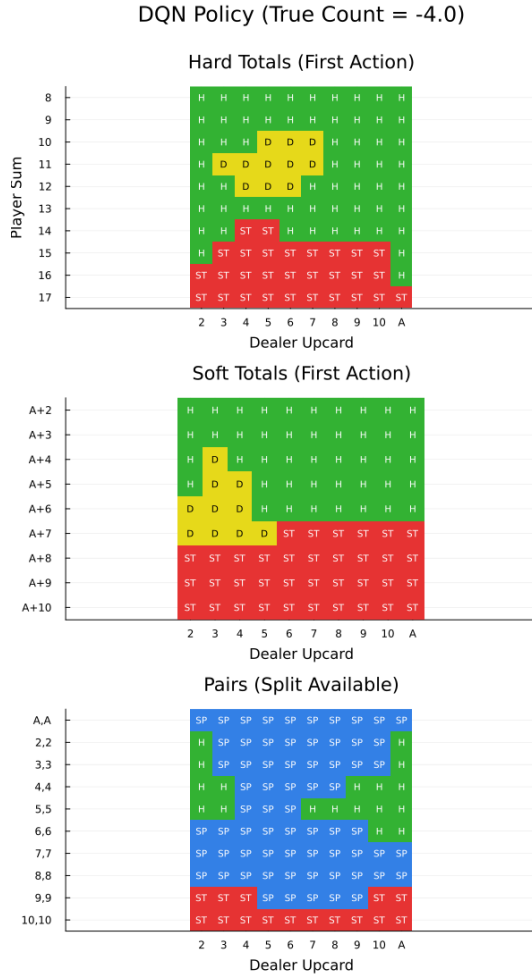


Fig. 3. Policy learned by the DQN algorithm when the count is low.

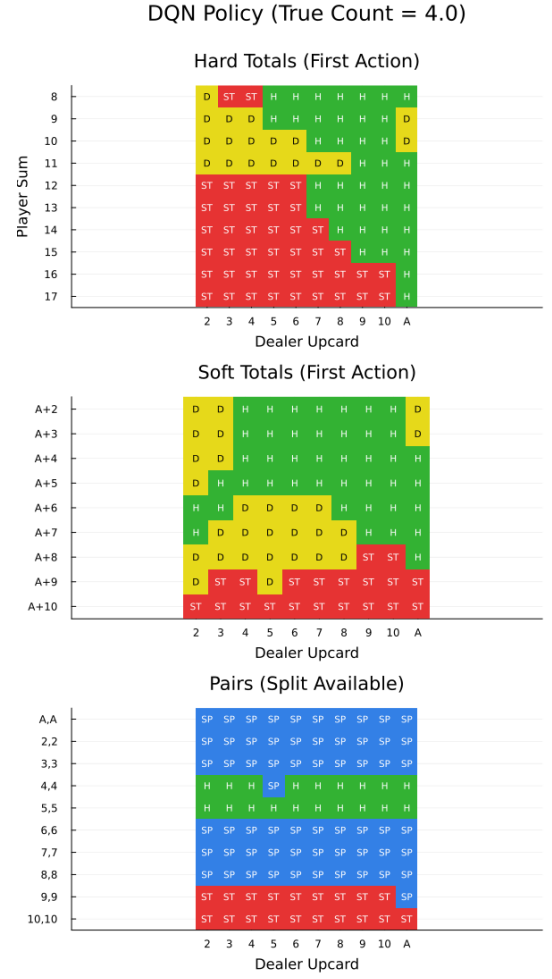


Fig. 4. Policy learned by the DQN algorithm when the count is high

D. Cross-Level Comparison

Table IV summarizes the improvement of each learned policy relative to its corresponding baseline.

TABLE IV
SUMMARY OF LEARNED POLICY IMPROVEMENTS

Setting	Baseline	Learned	Δ
1-Deck (Q-Learning)	-0.0886	-0.0757	+0.0129
6-Deck (DQN, Hi-Lo)	-0.1192	-0.0781	+0.0411

The larger improvement in the 6-deck setting is notable. The baseline suffers more from the increased deck count, but the DQN agent partially compensates by leveraging the true count to modulate its strategy based on shoe favorability. This supports the central hypothesis of this work: deck-aware state representations yield meaningful performance gains, and the benefit increases as the environment more closely mirrors real casino conditions.

VI. CONCLUSION

Our investigation demonstrates that reinforcement learning agents can effectively leverage the non-stationary nature of a

finite-deck shoe to improve performance over traditional static strategies.

A. Performance Analysis

In the 6-deck casino setting, the DQN agent achieved a mean return of -0.0781, a significant improvement over the dealer-mirroring baseline of -0.1192.

B. Deck-Awareness

The integration of the Hi-Lo true count as a state feature allowed the DQN to generalize deck favorability without the intractable overhead of tracking individual card counts. This suggests that "deck-aware" policies can meaningfully reduce the house edge even in multi-deck environments.

C. Limitations and Future Work

While the DQN outperformed the baseline, its performance was still negative, reflecting the inherent house edge of blackjack. Future work could involve more complex state representations, such as the Zen Count or Omega II systems, or utilizing a Recurrent Neural Network (RNN) to learn the deck's latent state directly from the sequence of cards observed without manual feature engineering. Also, allowing the agent to choose wager size could improve the simulation in the future. Currently the bet is always set at \$1, but allowing the agent to increase or decrease their bet size based on the deck's favorability.

D. Algorithm Implementations

Both the Q-Learning and DQN algorithms were implemented by the authors, not used off the shelf. Julia's Flux package was used for neural networks for DQN.

E. Differences From Proposal

This paper slightly differs from the original proposal. The first level of implementing a simple heuristic "dealer based" policy remains the same. The second level differs since a single deck implementation is now used, instead of the simpler infinite deck assumption outlined in the proposal. This increased the complexity of this level but helped simulate real casino practices better. The third level was also changed. After more research, it was decided to not model the problem as a POMDP where the dealer's down card is partially observable, but leave this card as part of the state of the deck. This allowed us to use DQN to solve the problem for larger shoe sizes.

VII. CONTRIBUTIONS AND RELEASE

A. Release

The authors grant permission for this report to be posted publicly.

B. Lucas Becker Contributions

Formulated problem as an MDP for both Q-Learning approach and DQN problem formulations. Implemented and tuned Q-Learning.

C. Theo Kishner Contributions

Implemented and tuned DQN algorithm.

REFERENCES

- [1] M. J. Kochenderfer, T. A. Wheeler, and K. H. Wray, *Algorithms for Decision Making*. Cambridge, MA: MIT Press, 2022.
- [2] Anonymous, "Beating blackjack: A reinforcement learning approach," Stanford University, Stanford, CA, Tech. Rep., 2020.
- [3] A. Finn, "Model-free Q-learning of blackjack," Stanford University, Stanford, CA, Tech. Rep., 2020.
- [4] Anonymous, "Deriving blackjack's basic strategy using reinforcement learning," *Acta Machina*, Mar. 2023. Available: <https://actamachina.com/posts/blackjack>
- [5] R. R. Baldwin, W. E. Cantey, H. Maisel, and J. P. McDermott, "The optimum strategy in blackjack," *J. Amer. Statist. Assoc.*, 1956.
- [6] M. Chiswick, "Monte Carlo RL and blackjack," Sep. 2020. [Online]. Available: <https://chisness.github.io/2020-09-21/monte-carlo-rl-and-blackjack>
- [7] Anonymous, "Reinforcement learning: Solving blackjack," *Towards Data Science*, 2020. [Online]. Available: <https://towardsdatascience.com/reinforcement-learning-solving-blackjack-5e31a7fb371f/>
- [8] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.

APPENDIX

All code for this project can be found publicly at the following Github repository.